

Experiment 4

Student Name: Balumuri Vivek Gowd
Branch: CSE - AIML
Semester: 4
Subject Name: DBMS

UID: 24BAI70001
Section/Group: 24AIT_KRG G1
Date of Performance: 4/2/26
Subject Code: 24CSH-298

Aim

To design and implement PL/SQL programs utilizing conditional control statements such as IF–ELSE, ELSIF, ELSIF ladder, and CASE constructs in order to control the flow of execution based on logical conditions and to analyse decision-making capabilities in PL/SQL blocks.

Software Requirements

- Database Management System:
 - PostgreSQL
- Database Administration Tool:
 - pgAdmin

Objectives

- Implement control structures in PL/SQL (IF-ELSE, ELSE-IF, ELSE-IF LADDER, CASE STATEMENTS in PL-SQL BLOCK).

Problem Statement

Develop and execute PL/SQL programs that demonstrate the use of conditional control statements. The programs should employ IF–ELSE, ELSIF, ELSIF ladder, and CASE statements to evaluate given conditions and control the flow of execution accordingly, thereby illustrating decision-making capabilities in PL/SQL blocks.

1. Problem Statement – IF–ELSE Statement

Write a PL/SQL program to check whether a given number is positive or non-positive using the IF-ELSE conditional control statement and display an appropriate message.

2. Problem Statement – IF-ELSIF-ELSE Statement

Write a PL/SQL program to evaluate the grade of a student based on the obtained marks using the IF-ELSIF-ELSE statement and display the corresponding grade.

3. Problem Statement – ELSIF Ladder

Write a PL/SQL program to determine the performance status of a student based on marks using an ELSIF ladder and display the appropriate result.

4. Problem Statement – CASE Statement

Write a PL/SQL program to display the name of the day based on a given day number using the CASE conditional statement.

Practical/Experiment Steps

- Control Structure Implementation: Designed multiple PL/SQL blocks to explore diverse conditional logic formats, including simple branching and multi-path evaluation.
- Logic Branching Analysis: Utilised IF-ELSE and ELSIF ladders to categorize numerical data into specific ranges, such as student grades and performance statuses.
- Selection Optimisation: Implemented the CASE statement as a streamlined alternative to multiple conditional checks for mapping discrete values like day numbers to names.
- Dynamic Messaging: Integrated variable-driven output strings to provide real-time feedback based on the evaluation of input conditions.
- Execution Flow Control: Validated the decision-making capabilities of the PL/SQL engine by testing various input scenarios to ensure the correct code path was activated.

Procedure

- Enabled the output server environment to ensure all procedural results would be visible in the console window.
- Constructed a basic IF-ELSE block to perform a binary check on a numerical variable for positive or non-positive properties.

- Developed an IF-ELSIF-ELSE structure to map student marks to specific letter grades based on defined percentage thresholds.
- Expanded the conditional logic into a comprehensive ELSIF ladder to categorise performance into tiers such as Distinction, First Class, and Pass.
- Implemented a CASE statement block to translate integer inputs into corresponding day names, including a default handler for invalid entries.
- Initialised diverse test values for each variable, such as negative numbers for sign checks and specific marks for grading, to verify logic accuracy.
- Nested the procedural logic within standard BEGIN...END; blocks to maintain structured programming principles.
- Executed each individual block sequentially and monitored the DBMS output console for the expected string concatenations.
- Verified that the output correctly reflected the logic branch associated with the assigned variable values and documented the results.
- Verified the console output against the manual calculations to ensure the logic and variables were handled correctly.

Input/Output Analysis

SQL Input Queries

```

DECLARE
NUM NUMBER:= -21;

BEGIN
  IF NUM > 0 THEN
    DBMS_OUTPUT.PUT_LINE('IT IS A POSITIVE NUMBER');
  ELSE
    DBMS_OUTPUT.PUT_LINE('IT IS A NON-POSITIVE NUMBER');
  END IF;
END;
```

Output

The screenshot shows the Oracle FreeSQL web interface. The top navigation bar includes the FreeSQL logo, a 'Worksheet' tab, and a 'Library' tab. The left sidebar contains a 'Navigator' panel with a tree view of database objects under the 'Human Resources (HR)' schema, including tables like HR.COUNTRIES, HR.DEPARTMENTS, HR.EMPLOYEES, HR.JOBS, HR.JOB_HISTORY, HR.LOCATIONS, and HR.REGIONS. The main workspace is titled '[SQL Worksheet]*' and contains a PL/SQL block:

```
1 DECLARE
2   NUM NUMBER:=10;
3
4   BEGIN
5     IF NUM>0 THEN
6       DBMS_OUTPUT.PUT_LINE('IT IS A POSITIVE NUMBER');
7     ELSE
8       DBMS_OUTPUT.PUT_LINE('IT IS A NON-POSITIVE NUMBER');
9     END IF;
10  END;
```

Below the code editor, the 'Script output' tab is active, displaying the execution results:

```
SQL> DECLARE
      NUM NUMBER:=10;
      BEGIN...
      Show more...

IT IS A POSITIVE NUMBER

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.004
```

The right sidebar features a 'Library' panel with a search bar and two tutorial cards. The first card is titled 'Creating Tables: Databases for...' and describes an introduction to creating tables in Oracle Database, created 8 years ago with 3 likes and 8.7K executions. The second card is titled 'Joining Tables: Databases for...' and describes an introduction to join types in Oracle Database, created 7 years ago with 8 likes and 1.8K executions.

This screenshot shows the same Oracle FreeSQL web interface, but with a different input value for the variable NUM. The PL/SQL block in the main workspace is:

```
1 DECLARE
2   NUM NUMBER:=21;
3
4   BEGIN
5     IF NUM>0 THEN
6       DBMS_OUTPUT.PUT_LINE('IT IS A POSITIVE NUMBER');
7     ELSE
8       DBMS_OUTPUT.PUT_LINE('IT IS A NON-POSITIVE NUMBER');
9     END IF;
10  END;
```

The 'Script output' tab shows the execution results for this block:

```
PL/SQL procedure successfully completed.
Elapsed: 00:00:00.004

SQL> DECLARE
      NUM NUMBER:=21;
      BEGIN...
      Show more...

IT IS A NON-POSITIVE NUMBER
```

The right sidebar remains the same, displaying the 'Library' panel with the same two tutorial cards.

SQL Queries Input

DECLARE

MARKS NUMBER:=68;

GRADE VARCHAR(1);

BEGIN

IF MARKS>=90 THEN

GRADE:='A';

ELSIF MARKS>=80 THEN

GRADE:='B';

ELSIF MARKS>=70 THEN

GRADE:='C';

ELSIF MARKS>=60 THEN

GRADE:='D';

ELSE

GRADE:='F';

END IF;

DBMS_OUTPUT.PUT_LINE('MARKS = '||MARKS||', GRADE = '||GRADE);

END;

Output

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a database structure for 'Human Resources (HR)' with tables like HR.COUNTRIES, HR.DEPARTMENTS, HR.EMPLOYEES, HR.JOBS, HR.JOB_HISTORY, HR.LOCATIONS, and HR.REGIONS. The main workspace shows an SQL script with the following code:

```
1 DECLARE
2 MARKS NUMBER:=98;
3 GRADE VARCHAR (1);
4
5 BEGIN
6 IF MARKS>=90 THEN
7 GRADE:='A';
8 ELSIF MARKS>=80 THEN
9 GRADE:='B';
10 ELSIF MARKS>=70 THEN
11 GRADE:='C';
12 ELSIF MARKS>=60 THEN
13 GRADE:='D';
14 ELSE
15 GRADE:='F';
16 END IF;
17
18 DBMS_OUTPUT.PUT_LINE('MARKS = '||MARKS||', GRADE = '||GRADE);
```

Below the script, the 'Script output' tab shows the execution results:

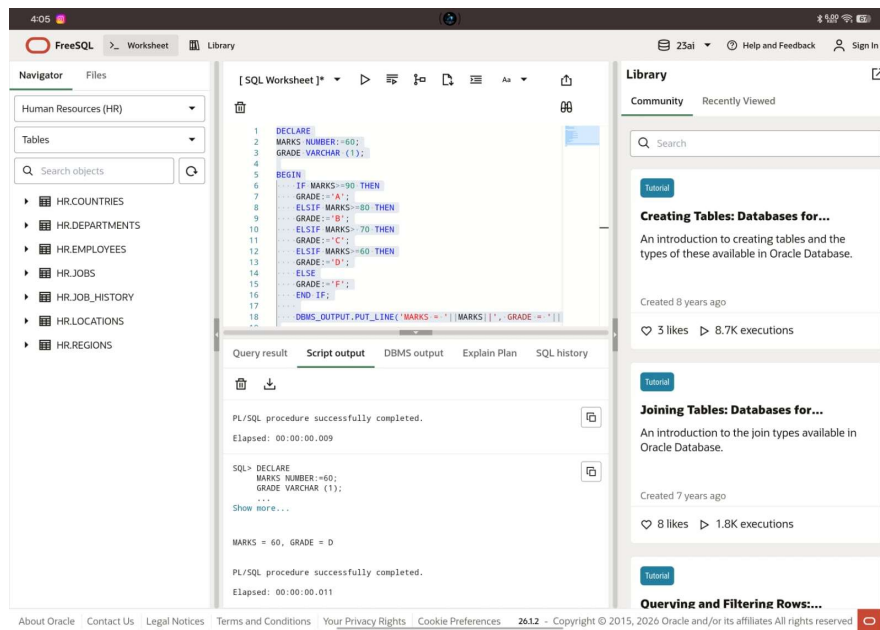
```
PL/SQL procedure successfully completed.
Elapsed: 00:00:00.005

SQL> DECLARE
MARKS NUMBER:=98;
GRADE VARCHAR (1);
...
Show more...

MARKS = 98, GRADE = A

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.009
```

On the right, a 'Library' pane displays a list of tutorials, including 'Creating Tables: Databases for...', 'Joining Tables: Databases for...', and 'Quervring and Filtering Rows...'. The bottom of the page contains a footer with links for 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', 'Cookie Preferences', and a copyright notice for 2015, 2020 Oracle and/or its affiliates.



SQL Queries Input

DECLARE

MARKS NUMBER:=58;

PERFORMANCE VARCHAR(20);

BEGIN

IF MARKS>=75 THEN

PERFORMANCE:='DISTINCTION';

ELSIF MARKS>=60 THEN

PERFORMANCE:='FIRST CLASS';

ELSIF MARKS>=50 THEN

PERFORMANCE:='SECOND CLASS';

ELSIF MARKS>=35 THEN

PERFORMANCE:='PASS';

ELSE

PERFORMANCE:='FAIL';

END IF;

DBMS_OUTPUT.PUT_LINE('MARKS = '||MARKS||' AND PERFORMANCE
= '||PERFORMANCE);
END;

Output

The image displays two screenshots of the Oracle SQL Developer interface, showing the execution of an SQL script. The top screenshot shows the script being executed with marks=38, resulting in 'PASS'. The bottom screenshot shows the script being executed with marks=88, resulting in 'DISTINCTION'. Both screenshots show the SQL script in the editor, the query result, and the DBMS output.

Top Screenshot:

- Navigator:** Human Resources (HR) | Tables | Search objects
- SQL Worksheet:**

```
1 DECLARE
2 MARKS NUMBER:=38;
3 PERFORMANCE VARCHAR (20);
4
5 BEGIN
6 IF MARKS>=75 THEN
7   PERFORMANCE:='DISTINCTION';
8 ELSEIF MARKS>=60 THEN
9   PERFORMANCE:='FIRST CLASS';
10 ELSEIF MARKS>=50 THEN
11   PERFORMANCE:='SECOND CLASS';
12 ELSEIF MARKS>=35 THEN
13   PERFORMANCE:='PASS';
14 ELSE
15   PERFORMANCE:='FAIL';
16 END IF;
17
18 DBMS_OUTPUT.PUT_LINE('MARKS = '||MARKS||', PERFORMANCE = '||PERFORMANCE||');
```
- Query result:** PL/SQL procedure successfully completed. Elapsed: 00:00:00.008
- DBMS output:** MARKS = 38, PERFORMANCE = PASS

Bottom Screenshot:

- Navigator:** Human Resources (HR) | Tables | Search objects
- SQL Worksheet:**

```
1 DECLARE
2 MARKS NUMBER:=88;
3 PERFORMANCE VARCHAR (20);
4
5 BEGIN
6 IF MARKS>=75 THEN
7   PERFORMANCE:='DISTINCTION';
8 ELSEIF MARKS>=60 THEN
9   PERFORMANCE:='FIRST CLASS';
10 ELSEIF MARKS>=50 THEN
11   PERFORMANCE:='SECOND CLASS';
12 ELSEIF MARKS>=35 THEN
13   PERFORMANCE:='PASS';
14 ELSE
15   PERFORMANCE:='FAIL';
16 END IF;
17
18 DBMS_OUTPUT.PUT_LINE('MARKS = '||MARKS||', PERFORMANCE = '||PERFORMANCE||');
```
- Query result:** PL/SQL procedure successfully completed. Elapsed: 00:00:00.008
- DBMS output:** MARKS = 88, PERFORMANCE = DISTINCTION

SQL Queries Input
DECLARE
DAYNUM NUMBER:=3;
DAYNAME VARCHAR(20);

BEGIN

```

DAYNAME:=CASE DAYNUM
WHEN 1 THEN 'SUNDAY'
WHEN 2 THEN 'MONDAY'
WHEN 3 THEN 'TUESDAY'
WHEN 4 THEN 'WEDNESDAY'
WHEN 5 THEN 'THURSDAY'
WHEN 6 THEN 'FRIDAY'
WHEN 7 THEN 'SATURDAY'
ELSE 'INVALID DAY'
END;

```

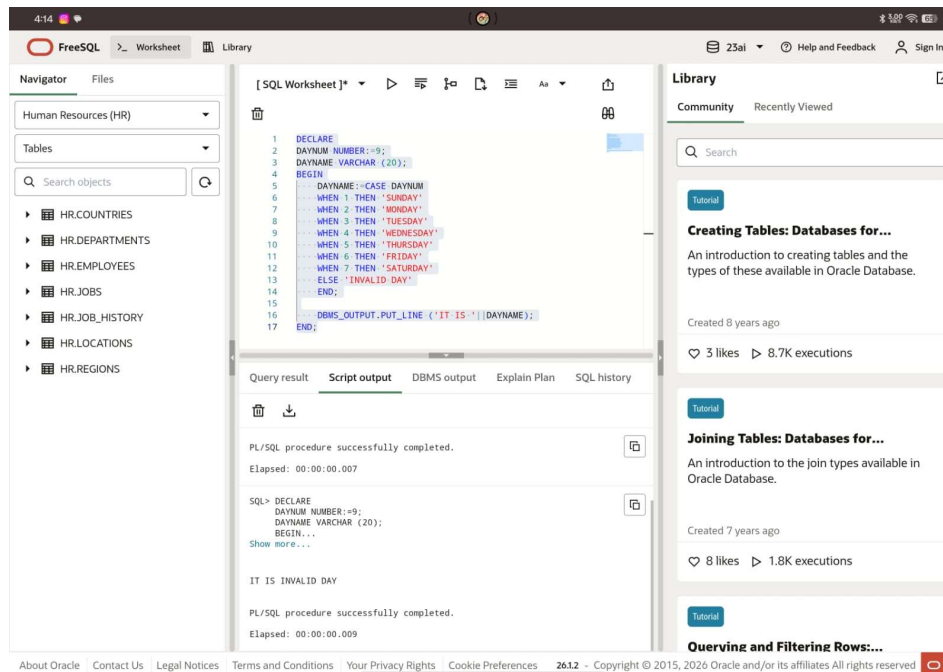
```

DBMS_OUTPUT.PUT_LINE('IT IS '||DAYNAME);
END;

```

Output

The screenshot displays the FreeSQL web application interface. On the left, a 'Navigator' pane shows a tree view of database objects under 'Human Resources (HR)', including tables like HR.COUNTRIES, HR.DEPARTMENTS, HR.EMPLOYEES, HR.JOBS, HR.JOB_HISTORY, HR.LOCATIONS, and HR.REGIONS. The main workspace shows a SQL script in a text editor with line numbers 1 through 17. The script declares a variable DAYNUM with a value of 3, and a variable DAYNAME using a CASE statement. The output pane at the bottom shows the execution results, including the text 'IT IS TUESDAY' and a message that the PL/SQL procedure completed successfully. On the right, a 'Library' pane displays a list of tutorials, including 'Creating Tables: Databases for...' and 'Joining Tables: Databases for...'. The bottom of the interface contains a footer with links to 'About Oracle', 'Contact Us', 'Legal Notices', 'Terms and Conditions', 'Your Privacy Rights', and 'Cookie Preferences', along with a copyright notice for 2015, 2026 Oracle and/or its affiliates.



Learning Outcomes

- Gained proficiency in using IF-ELSE, ELSIF ladders, and CASE statements to control program execution flow.
- Evaluated data variables to automate specific outcomes, such as student grading or performance status.
- Using CASE statements as a streamlined method for mapping discrete values like day numbers to names.
- Skills in setting logical thresholds to categorize raw numerical marks into descriptive classifications